

CRYME

Cryptographic Migration Engineering

Ein simulationsbasiertes Oracle zur Orchestrierung der PQC-Migration

mittels digitaler Zwillinge und Graphdatenbanken

Erick Zeiler · Masterprojekt Informatik · Hochschule RheinMain · Betreuung: Prof. Dr. Marc Stöttinger, Prof. Dr. Bodo Igler

1 Motivation & Problemstellung

Quantencomputer bedrohen klassische asymmetrische Kryptografie. Der Umstieg auf **Post-Quanten-Kryptografie (PQC)** ist daher unausweichlich – aber er bedeutet weit mehr als den bloßen Austausch von Algorithmen.

Reale Systeme enthalten **verborgene kryptografische Abhängigkeiten** zwischen Schlüsseln, Zertifikaten, Protokollen und Komponenten (z.

B. gemeinsam genutzte Schlüssel, Trust-Ketten, Protokoll-Bindungen – nicht nur TLS). Wird ein einzelnes Asset isoliert migriert, ohne diese Abhängigkeiten zu berücksichtigen, droht ein **Systemausfall**.

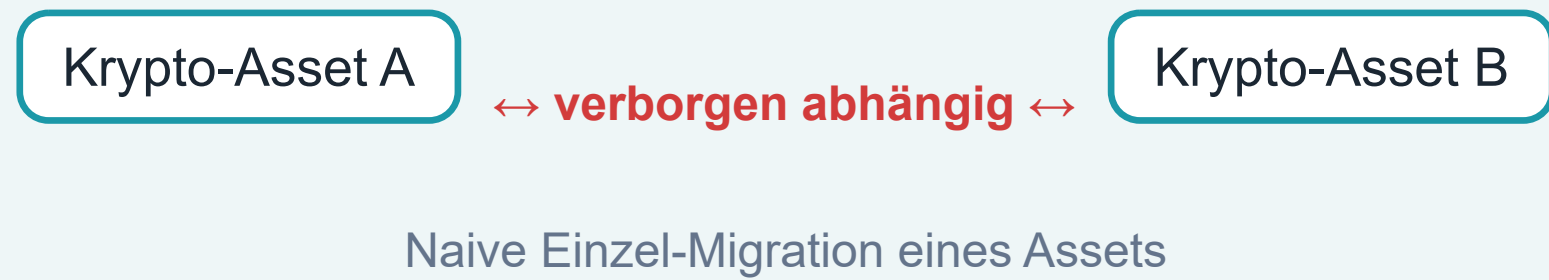


Abb. 1: Eine isolierte Migration verletzt versteckte Abhängigkeiten und bricht das System.

2 Zielsetzung

Entwicklung eines **generischen, systemunabhängigen Simulations-Frameworks („Oracle“)**, das Migrationsstrategien beliebiger kryptografischer Landschaften **präventiv** validiert, **bevor** sie auf dem realen System ausgeführt werden. Ziele im Überblick:

- Abbildung realer Systeme als vereinfachte **digitale Zwillinge**
- Automatische** Erkennung versteckter Abhängigkeiten
- Bestimmung einer **korrekten Migrations-Reihenfolge**
- Präventives **Blockieren & Zurückrollen** fehlerhafter Schritte
- Visualisierung des Migrationsfortschritts (PoC)

3 Lösungsansatz: Das Oracle als Gatekeeper

Das Oracle arbeitet nach dem Prinzip „**prüfen vor ausführen**“ (deny-by-default): Kein Migrationsschritt gilt automatisch als sicher – jeder Schritt muss zuerst gegen **alle** bekannten Abhängigkeiten validiert werden, sonst wird er abgewiesen.

Dynamic Dependency Discovery versteckte strukturelle Abhängigkeiten zwischen Assets (z.

B. gemeinsam genutzte Schlüssel oder Trust-Beziehungen) werden zur Laufzeit erkannt und zwingend zu **Co-Migrations-Clustern** zusammengefasst.

Temporal Barrier Enforcement ein PQC-Hybrid-System wird erst aktiviert, wenn alle zwingenden

Vorbedingungen erfüllt sind (keine Out-of-Order-Zustände).



Abb. 2: Verarbeitungs-Pipeline von der Modellierung bis zum geprüften Plan.

4 Architektur & Umsetzung

Die realen Anwendungssysteme werden als **vereinfachte digitale Abbilder (Digital Twins)** modelliert und über **YAML** nach dem Prinzip „Infrastructure as Code“ beschrieben und verwaltet.

Warum graphbasiert? Kryptografische Assets und ihre Abhängigkeiten bilden von Natur aus ein **Netzwerk**. Eine **Neo4j-Graphdatenbank** bildet Assets als **Knoten** und Abhängigkeiten als **Kanten** ab. So lassen sich Co-Migrations-Cluster durch Graph-Traversierung finden und eine korrekte Reihenfolge per topologischer Sortierung bestimmen – Aufgaben, die in tabellarischen Strukturen nur schwer abbildbar sind.

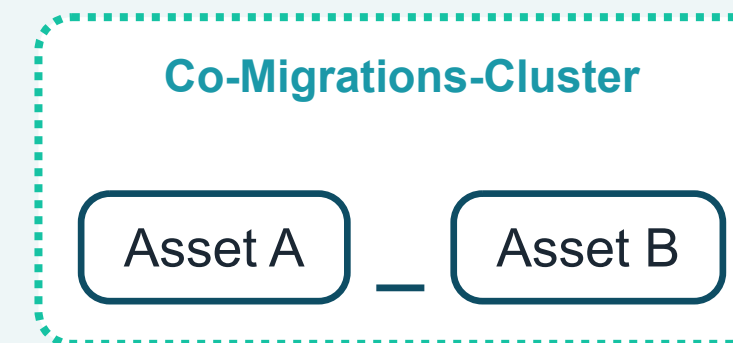


Abb. 3: Knoten = Assets, Kanten = Abhängigkeiten. Verbundene Knoten werden gemeinsam migriert.

5 Ergebnisse: Live-Simulation

Der Proof of Concept blockiert fehlerhafte Schritte präventiv und rollt sie transaktional zurück; konsistente Cluster werden erfolgreich migriert. Die folgende Ausgabe zeigt dies an einem konkreten Beispielsystem:

```
root@cryme-oracle $ node cryme show tree
PQC Migration Tree:
[Step 0] INIT (system_start)
├─ [Step 1] FAILED (Policy Denied: Webserver.Key)
├─ [Step 2] SUCCESS (Co-Migrated Webserver.Key)
├─ [Step 3] ABORTED (Redundant State Detected)
├─ [Step 4] SUCCESS (Migrated Webserver.Cert)
└─ [Step 5] SUCCESS (Migrated Webserver)

root@cryme-oracle $ node cryme show system
=====
ID | Component | Status | Algorithm
-----
32 | Client_Browser | migrated | MLKEM768
27 | Webserver_Classic | migrated | ML-DSA-65
26 | Webserver_Classic | migrated | TLS1.3
```

Analyse: Step 1 wird wegen einer verletzten Abhängigkeit abgewiesen; Step 2 zeigt die erfolgreiche Co-Migration des erkannten Clusters.

6 Fazit & Ausblick

Der PoC zeigt, dass sich riskante PQC-Migrationsschritte **vor** der Ausführung simulativ erkennen und abfangen lassen. Versteckte Abhängigkeiten werden automatisch zu Clustern gebündelt und in eine konsistente Reihenfolge gebracht.

Ausblick (langfristige Vision): Auf dieser Grundlage könnte künftig eine nachvollziehbare, weitgehend automatisierte Planung komplexer unternehmensweiter PQC-Iterationen aufgebaut werden – die vollständige Absicherung realer Großsysteme bleibt Gegenstand weiterführender Arbeiten.